04 分治法

Part I Foundations

本章定位

分治的正確性靠完整分類與遞迴契約；本章以最大子陣列及Strassen七乘積建立算法，再用同一常數的歸納、逐層求和與完整主定理證明分析時間，明列取整、對數邊界與不等比例條件。

前置知識

遞迴、陣列區間、矩陣乘法的基本索引與歸納法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用分治計算最大連續子陣列。
- 解釋 Strassen 矩陣乘法的分塊與次立方成本。
- 使用代入法、遞迴樹與主定理分析遞迴式。
- 在子問題大小不相等時，辨識 Akra–Bazzi 的外部適用條件並完成積分。
- 4.1 The maximum-subarray problem
- 4.2 Strassen’s algorithm for matrix multiplication
- 4.3 The substitution method for solving recurrences
- 4.4 The recursion-tree method for solving recurrences
- 4.5 The master method for solving recurrences
- 4.6 Proof of the master theorem

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分治的規格與成本要分開建立

分治（divide and conquer）先把輸入拆成更小的同類問題，遞迴解決，再合併答案。正確性由「子問題答案足以涵蓋原問題」與歸納建立；成本則把每次呼叫的分割、合併、配置、複製逐項計入。

本章以1-based陣列及包含端點的區間為主。時間採精確實數／整數的單位成本算術模型；若整數位元長度增長，乘加不能仍無條件算O(1)。矩陣乘法先用精確算術證明，不把代數恆等式當成浮點誤差保證。

遞迴等式 $T(n) = \sum_i T(n_i) + f(n)$ 描述完整成本；若只知 $T(n) \le \sum_i T(n_i) + f(n)$，通常只能推出上界。還需基例、每個 $n_i < n$ 的終止條件及分割的實際成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.1 The maximum-subarray problem

給n≥1個數A[1..n]，求一個非空、連續區間[i,j]，使 $\sum_{k=i}^{j} A[k]$ 最大，回傳(i,j,sum)。相同最大值可有多個答案；除非另訂規則，只需回傳其中一個。

價格P0,⋯,Pn若必須先買後賣，令Ai=Pi−P(i−1)，則 $P_j - P_{i-1} = \sum_{k=i}^{j} A_k$。故區間i..j對應第i−1天買、第j天賣。若允許不交易，則可另選空區間、收益0；全負輸入時兩種規格答案不同。

逐一枚舉起點，再延長終點並累加，可用Θ(n^2)時間、O(1)額外空間求解；若每個區間又從頭加總，才會變成Θ(n^3)。前綴和也可把一次區間求和降到O(1)，但仍有Θ(n^2)個候選區間。

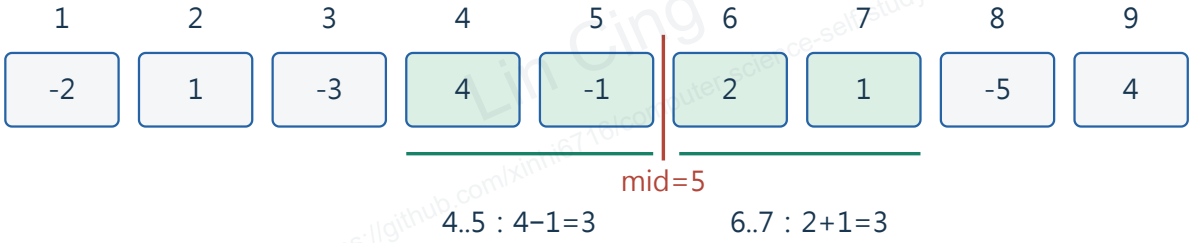
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨中線必須由 suffix 與 prefix 組成

對low<high，令mid=⌊(low+high)/2⌋。任一非空區間恰屬三類：完全在low..mid、完全在mid+1..high、或i≤mid<j。第三類必包含mid與mid+1，可唯一拆成左半的suffix i..mid與右半的prefix mid+1..j。

兩側端點可獨立選擇，故最佳跨線和等於「最佳非空左suffix和」加「最佳非空右prefix和」。不需要枚舉每個(i,j)配對；分別掃描兩側即可。只取左右兩半各自的最大子陣列會出錯，因它們未必接到中線。

索引與非空跨線區間：A[4..7] 是本輪答案



跨線和 3+3=6；左suffix與右prefix必須緊接中線。

跨線掃描的完整程序

```
CROSS(A, low, mid, high):
    left = -infinity; total = 0
    for i = mid downto low:
        total += A[i]
        if total > left: left = total; best_i = i
    right = -infinity; total = 0
    for j = mid+1 to high:
        total += A[j]
        if total > right: right = total; best_j = j
    return (best_i, best_j, left+right)
```

左迴圈每次更新後，total恰為當前i..mid的和，left是已掃過所有suffix的最大和；初始化沒有候選，−∞不會錯選空區間。每輪新增唯一一個候選並取最大，因此不變量成立。終止時所有左suffix均已納入；右側同理。掃描總次數(mid−low+1)+(high−mid)=high−low+1。

最大子陣列的遞迴與一般證明

```
MAX-SUBARRAY(A, low, high):
    if low == high: return (low, high, A[low])
    mid = floor((low+high)/2)
    L = MAX-SUBARRAY(A, low, mid)
    R = MAX-SUBARRAY(A, mid+1, high)
    C = CROSS(A, low, mid, high)
    return a tuple with largest sum among L, R, C
```

對區間長度強歸納：單元素只有自身；較長時兩個遞迴區間均非空且較短，故由歸納假設求得前兩類最大值，CROSS已證求得第三類最大值。三類互斥且完整，三者取最大即全域最大，長度嚴格縮小也保證終止。

$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) = \Theta(n \lg n)$。順序遞迴使用O(lg n)stack，每層只存常數個索引與和；若額外複製子陣列，需另計配置與活躍空間，不能沿用此空間界。

九元素數例：每次跨線比較都可核對

A=[−2,1,−3,4,−1,2,1,−5,4]，low=1、high=9、mid=5。

左索引i	A[i..5]累計	目前最大與起點
5	−1	−1，i=5
4	3	3，i=4
3	0	3，i=4
2	1	3，i=4
1	−1	3，i=4

右側j=6,7,8,9的prefix和依序2、3、−2、2，所以最佳是6..7、和3。跨線區間4..7和6；左半最佳4..4和4，右半最佳9..9和4，故本輪選(4,7,6)。全負例[−8,−2,−5]會選(2,2,−2)，不會回0。

線性方法的狀態與不變量

令end_i為必須以i結尾的最佳非空區間和。這種區間要麼只有Ai，要麼由i−1結尾的最佳區間接上Ai，因此 $end_i = \max(A_i, end_{i-1} + A_i)$。全域best_i=max(best_(i−1),end_i)，初值end_1=best_1=A1。

每輪若選Ai便重設起點i；否則保留上一個起點。best改善時保存該起點及終點i。歸納可證end與best分別是定義中的兩個最優值，故Θ(n)時間、O(1)額外空間即可回傳區間。

九元素例end為[−2,1,−2,4,3,5,6,1,5]，best為[−2,1,1,4,4,5,6,6,6]，最佳4..7。分治不是此問題的最快方法，但它展示「類別覆蓋＋合併規格」如何構成正確性證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.2 Strassen’s algorithm for matrix multiplication

對n×n矩陣，$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$。三層迴圈先令Cij=0，再依k加入乘積。內圈不變量是已完成前k項總和；n^2個輸出各n次乘加，故Θ(n^3)。這只是這個程序的成本，並未證明所有矩陣乘法都需Ω(n^3)。

把A、B分成2×2區塊，令A=[[a,b],[c,d]]、B=[[e,f],[g,h]]，各字母是n/2階矩陣。區塊乘法給C11=ae+bg、C12=af+bh、C21=ce+dg、C22=cf+dh。順序必須保留；區塊矩陣一般不交換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

八次遞迴乘法的完整成本

程序輸入為同階方陣，先補零至2的冪次階數，再取回原尺寸的結果。每個遞迴呼叫由輸入view的rows取得自己的階數，不另傳原矩陣的n。

```
BLOCK-MULTIPLY(A,B):
    n = A.rows
    if n == 1: return [[A[1,1]*B[1,1]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    C11 = BLOCK-MULTIPLY(a,e)+BLOCK-MULTIPLY(b,g)
    C12 = BLOCK-MULTIPLY(a,f)+BLOCK-MULTIPLY(b,h)
    C21 = BLOCK-MULTIPLY(c,e)+BLOCK-MULTIPLY(d,g)
    C22 = BLOCK-MULTIPLY(c,f)+BLOCK-MULTIPLY(d,h)
    return the four blocks assembled into C
```

索引view只傳起點、stride與階數，分割可$O(1)$；若真的複製區塊，是$\Theta(n^2)$，仍和四個區塊加法同階。遞迴式為$8T(n/2)+\Theta(n^2)$，基例$\Theta(1)$，解$\Theta(n^3)$。

常數個矩陣加法可合成$\Theta(n^2)$，但不能把$8T(n/2)$改成$T(n/2)$：這個8每層都增殖，決定葉數n^3。對固定一次呼叫吸收常數，與改變整棵遞迴樹的分支數，是不同操作。

Strassen 的七個乘積全部列出

以下七個式子以遞迴STRASSEN計算；其輸入的加減皆是逐元素區塊運算。

$$
\begin{aligned}
M_1 &= (a+d)(e+h), & M_2 &= (c+d)e,\\
M_3 &= a(f-h), & M_4 &= d(g-e),\\
M_5 &= (a+b)h, & M_6 &= (c-a)(e+f),\\
M_7 &= (b-d)(g+h).
\end{aligned}
$$

回收結果為 $C_{11}=M_1+M_4-M_5+M_7$，$C_{12}=M_3+M_5$，$C_{21}=M_2+M_4$，$C_{22}=M_1-M_2+M_3+M_6$。n=1直接做一個純量乘法；n>1依此分割、計七乘積並回收四塊，構成完整遞迴算法。這套M編號是本章的定義，不與其他教材的P編號混用。

四個輸出區塊為何正確

只需分配律與加法消去，沒有交換矩陣乘法的次序：

$$\begin{aligned}
C_{11} &= (ae + ah + de + dh) + (dg - de) \\
&\quad - (ah + bh) + (bg + bh - dg - dh) = ae + bg, \\
C_{12} &= (af - ah) + (ah + bh) = af + bh, \\
C_{21} &= (ce + de) + (dg - de) = ce + dg, \\
C_{22} &= (ae + ah + de + dh) - (ce + de) \\
&\quad + (af - ah) + (ce + cf - ae - af) = cf + dh.
\end{aligned}$$

基例純量乘法正確；若七個較小乘積由歸納假設正確，上述恆等式回收到AB的全部四塊，完成一般正確性證明。需要可加、可減及分配律；只有加法、沒有減法的一般semiring不能直接使用這七式。

七乘積的數值與四塊回收

A=[[2,−1],[3,4]]，B=[[5,2],[−2,1]]，所以a=2,b=−1,c=3,d=4,e=5,f=2,g=−2,h=1。

乘積	代入	值
M1	(2+4)(5+1)	36
M2	(3+4)5	35
M3	2(2−1)	2
M4	4(−2−5)	−28
M5	(2−1)1	1
M6	(3−2)(5+2)	7
M7	(−1−4)(−2+1)	5

四塊依序36−28−1+5=12、2+1=3、35−28=7、36−35+2+7=10。直接乘法也得[[10+2,4−1],[15−8,6+4]]=[[12,3],[7,10]]。

七乘積的數值與四塊回收 (續)

同一輸入的七乘積：四個回收位置不能互換

A

2	-1
3	4

B

5	2
-2	1

M_1=36；M_2=35

M_3=2；M_4=−28

M_5=1；M_6=7

M_7=5

C

12	3
7	10

C_{11}=36−28−1+5=12；C_{12}=2+1=3

C_{21}=35−28=7；C_{22}=36−35+2+7=10

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strassen 的時間、補零與記憶體

分割view為O(1)，準備七乘積輸入共10次區塊加減，回收四塊共8次區塊加減。因此 $T(n) = 7T(n/2) + \Theta(n^2)$，$T(1) = \Theta(1)$，解 $\Theta(n^{\lg 7})$，$\lg 7 \approx 2.80735$。加減的常數較大，小矩陣不一定較快；可在固定閾值改用普通乘法，不改漸近指數。

任意正n可補零到m=2^⌈lg n⌉，n≤m<2n。補零矩陣左上乘積仍是AB，取回n×n即可；m^(lg7) <7n^(lg7)，所以同階。n=0直接回空矩陣，不遞迴。

深度優先、每個呼叫只保留常數個當層區塊，依次完成七個子呼叫並適時釋放暫存時，峰值額外空間滿足 $S(n) \le S(n/2) + O(n^2) = O(n^2)$，另有O(lg n)stack。若同時保留全部子問題工作區，空間遞迴不同，不能聲稱同一界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.3 The substitution method for solving recurrences

代入法先猜候選界，再以強歸納找出同一組固定常數，使所有較小輸入的假設能推出當前輸入。不能在每層重新選一個較大的常數。

例$T(n)=2T(\lfloor n/2\rfloor)+n$，$T(1)=1$，$n\geq 2$。欲證$T(n)\leq cn\lg n$：對$n\geq 4$，$m=\lfloor n/2\rfloor\geq 2$，代入得$2cm\lg m+n\leq cn\lg(n/2)+n=cn\lg n-(c-1)n\leq cn\lg n$，取$c\geq 1$。

但$n=1$右側為0，不能拿來當歸納基。改以$n=2$、3為基：$T2=4$、$T3=5$，$c\geq 2$可覆蓋；$n\geq 4$的child均≥ 2，證明閉合。也可改猜$cn(\lg n+1)$，一併涵蓋$n=1$。漸近結論不容許漏掉實際會被呼叫、又未被基例處理的小輸入。

同一常數的下界與強化假設

對二冪n，$T(n)=2T(n/2)+n$、$T1=1$，代入候選$T(n)\geq dn\lg n$得$T(n)\geq dn\lg n+(1-d)n$，取$0<d\leq 1$便成立；配基例得到$\Omega(n\lg n)$。非二冪可用後面的取整樹證明延伸，與上界合成$\Theta(n\lg n)$。

另一例$T(n)=T(\lfloor n/2\rfloor)+T(\lceil n/2\rceil)+1$，$T1=1$。直接假設$T(n)\leq cn$會得到$cn+1$，不能閉合；這不表示線性猜測錯。強化為$T(n)\leq cn-d$，代入得$cn-2d+1\leq cn-d$，只需$d\geq 1$、$c\geq d+1$覆蓋$T1$。實際$c=2,d=1$還給精確解$2n-1$。

若把「$cn+n$仍是$O(n)$」當成第一個遞迴的線性證明，就是偷偷把c改成$c+1$，並未推出原假設；這種錯誤會把$n\lg n$誤證成線性。

換變數與真正縮小的條件

對T(n)=2T(√n)+lg n，先在n=2^(2^k)上定義，T2=Θ(1)。令m=lg n、S(m)=T(2^m)，則S(m)=2S(m/2)+m，得到S(m)=Θ(m lg m)，還原為T(n)=Θ(lg n・lg lg n)。不能把S的輸入m直接當成原n。

固定平移也要檢查終止。例如T(n)=2T(n/2+17)+n在n>34才有child<n；設n=34+x、U(x)=T(34+x)，得到U(x)=2U(x/2)+x+34。以足夠大的常數區間為基，樹求和為Θ(x lg x)=Θ(n lg n)。整數版本再保留floor誤差；不能只給T1而讓n=34反覆呼叫自己。

上述換元例的域已明列；把二冪上的證明推到任意n，仍需另外的取整、單調比較或適當基例，不是自動成立。

4.4 The recursion-tree method for solving recurrences

樹節點記的是本次呼叫的非遞迴成本f(size)，child才記遞迴呼叫；若在每個節點放完整T(size)後又加子樹，就會重複計費。要另外算深度、每層節點數及葉成本。

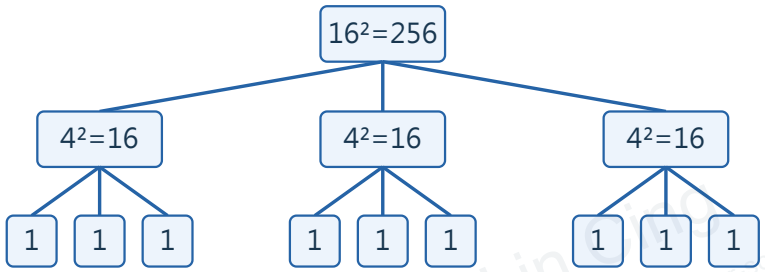
取原創遞迴T(n)=3T(n/4)+n²、T1=1，n=4^L。深度j有3^j個節點，每個成本(n/4^j)²，故層成本n²(3/16)^j。內層j=0..L−1，葉有3^L=n^(log₄3)，所以

$$T(n) = n^2 \sum_{j=0}^{L-1} (3/16)^j + n^{\log_4 3} = \Theta(n^2).$$

上界來自收斂幾何和≤16/13及葉成本≤n²，下界來自根的n²。n=16的實際層成本為256、48、9，總313；葉層9已是基例成本，不能再加9次f(1)。

4.4 The recursion-tree method for solving recurrences （續）

三個 n/4 子問題：每層成本乘 3/16，而不是 3/4



深度0：1×256=256

深度1：3×16=48

深度2：9×1=9

T(16)=256+48+9=313；最下排已是基例，不重複計費。

不等分割：先界深度，再界每層質量

令整數n≥2，a=⌊n/3⌋、b=n−a；對足夠大n用T(n)=T(a)+T(b)+cn，常數小輸入直接處理。兩個child大小和等於n，所以尚未終止的某一層，其輸入總量至多n，層成本至多cn。

最長路每步至多2n/3+1，深度O(lg n)，葉共O(n)，所以總成本O(n lg n)。下界不能只說「每層都是n」直到最深葉，因短路徑會提早終止。前⌊log₃(n/K)⌋層尚無任何路徑進入固定基例（選K足夠大吸收取整），這些完整層每層成本cn，故Ω(n lg n)。

若非遞迴項只有O(n)而非Θ(n)，上述上界仍成立，但不能由此推出相同下界；例如合併成本實為0時，葉成本可能只給Θ(n)。

4.5 The master method for solving recurrences

設a≥1、b>1固定，T有正且有界的常數大小基例，f≥0且最終為正，遞迴T(n)=aT(n/b)+f(n)。令p=log_b a。先用精確冪理解，再依下一節的條件處理整數取整。

情形	必須核對的條件	結論
1	存在固定ε>0，f(n)=O(n^(p−ε))	Θ(n^p)
2	f(n)=Θ(n^p)	Θ(n^p lg n)
3	固定ε>0，f(n)=Ω(n^(p+ε))；且af(n/b)≤cf(n)，固定c<1、充分大n成立	Θ(f(n))

f比n^p小或大，並不足以取代多項式分離；ε必須與n無關。情形3的正則性限制向下各層總成本，使它們可由根成本支付。a若是實數，可視為遞迴權重；描述實際a個呼叫時a必為整數。

同一張表的完整判定數例

遞迴的(a,b,f)	p與必要核對	結果
(9,3,n)	p=2，差n^1，情形1	Θ(n^2)
(1,3/2,1)	p=0，情形2	Θ(lg n)
(3,4,n lg n)	p≈0.7925；選ε=0.1；3f(n/4)/f(n)≤3/4，情形3	Θ(n lg n)
(8,2,n^2)	p=3，ε=1	Θ(n^3)
(7,2,n^2)	p=lg7，任取0<ε≤lg7−2	Θ(n^(lg7))

T=2T(n/2)+n lg n不屬經典三情形：lg n比任何固定n^ε小，f/n不是多項式分離；但每層求和仍可解。工具不適用不代表遞迴沒有解。

對數邊界必須真的求和

在精確冪n=b^L、固定常數大小基例下，若 $f(n)=\Theta(n^p(\lg n)^k)$，任何固定實數k都可由層和分析：由葉端令r=L−j，層成本為Θ(n^p r^k)，所以

$$T(n)=\begin{cases}\Theta(n^p(\lg n)^{k+1}) & k>-1,\\ \Theta(n^p\lg\lg n) & k=-1,\\ \Theta(n^p) & k<-1.\end{cases}$$

k>−1由積分比較 $\sum_{r=1}^{L} r^k=\Theta(L^{k+1})$；k=−1是調和和Θ(lg L)；k<−1收斂到正常數，加上葉成本仍Θ(n^p)。原書擴充練習的k≥0是其中一部分。此處在n大於固定基例時使用log，避開lg1=0的負次方。

例如2T(n/2)+n/lg n為Θ(n lg lg n)，不是Θ(n)；9T(n/3)+n²lg²n為Θ(n²lg³n)。

正則性缺失的明確反例

只看n=2^L、T1=1，令f(n)=n²當L偶數、f(n)=n³當L奇數。每個n都有f(n)≥n²，對a=b=2、p=1已滿足多項式分離，但不能得T=Θ(f)。

在偶數L且n≥4時，子問題n/2的指數奇數，因此 $T(n)\ge 2f(n/2)=n^3/4$。當前f(n)=n²，故T(n)/f(n)≥n/4無界。相同計算給2f(n/2)/f(n)=n/4，正是正則性失敗。

這不是反駁主定理，而是展示缺條件後結論會失效。也不能以「floor只差1」推任意震盪f值接近；取整的實際自變數仍要放進所用條件。

4.6 Proof of the master theorem

先令n=b^L，基例T1=t>0。反覆展開恰L次，或按樹深度歸納，可得精確式

$$T(n) = a^L T(1) + \sum_{j=0}^{L-1} a^j f(n/b^j) = tn^p + \sum_{j=0}^{L-1} a^j f(n/b^j).$$

葉成本tn^p提供所有情形的基本正下界。情形1若f(x)≤Cx^(p−ε)，層成本≤Cn^(p−ε)(b^ε)^j，幾何比b^ε>1，因此總和O(n^(p−ε)(b^ε)^L)=O(n^p)。固定小x不滿足上界的末幾層也只有O(n^p)，可由有限個f值吸收；加葉成本得Θ(n^p)。

幾何和是往葉端增加，不是遞減。例如a=b=2、ε=1/2，比為√2>1。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

主定理等高與根主導的兩個證明

情形2：a=b^p使每層 $a^j\Theta((n/b^j)^p) = \Theta(n^p)$。有L−O(1)=Θ(lg n)個大於固定基例的層，雙向界相加得Θ(n^p lg n)，葉成本被其涵蓋。

情形3：反覆使用 $af(x/b) \le cf(x)$，在大小仍充分大的每層得到 $a^j f(n/b^j) \le c^j f(n)$。因此這些內層總和≤f(n)/(1−c)。剩下固定大小層及葉的總成本O(n^p)；由f(n)=Ω(n^(p+ε))，它們也是O(f(n))。根本身f(n)給下界，所以Θ(f(n))。

固定小輸入的總成本不是整棵樹的O(1)：可能有Θ(n^p)個小節點。本證明將它們明確併入葉階，而不是漏算。三情形均同時交付上、下界。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

取整的誤差不會逐層線性累積

每條路徑令q0=n，q(j+1)是⌊qj/b⌋或⌈qj/b⌉。兩者皆滿足 $|q_{j+1} - q_j/b| \le 1$。若C≥b/(b−1)，由歸納

$$|q_j - n/b^j| \le C,$$

因下一層誤差至多C/b+1≤C。選固定基例範圍q≤n0，n0>C+2且夠大使child<q；則所有路徑進基例的深度均為log_b n+O(1)。在距葉端至少固定層數的位置，n/b^j≥2C，所以實際qj與n/b^j在固定倍數內。

實際a個child時，各路徑深度只差常數，故葉數Θ(a^(log_b n))=Θ(n^p)，所有固定大小底層成本也是O(n^p)。a=1時是一條路、常數個底層節點；公式仍成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從精確冪推到所有整數的成本條件

情形1：在離基底固定層數以上，qj與n/b^j同階，即使p−ε為負也有qj^(p−ε)=Θ((n/b^j)^(p−ε))；套f(qj)的多項式上界，得到與精確冪相同的增加幾何和。底層O(n^p)另計。

情形2：同樣有f(qj)=Θ((n/b^j)^p)，每個完整的大規模層成本Θ(n^p)，且有Θ(lg n)層；不完整的固定個底層不改此界。對數擴充對遠離固定基例的層也以r=距底層深度比較，log qj與r同階，得到前述三種對數和。

情形3必把正則性施於實際child：若可能有floor或ceiling，充分條件是 $a\max\{f(\lfloor x/b\rfloor), f(\lceil x/b\rceil)\} \le cf(x)$，c<1。如此逐層總成本仍以c倍收縮；加O(n^p)底層並用多項式分離即可。只證qj相差常數，對任意f不能省略這個條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正則性也能推出多項式分離

在精確冪上，若af(x/b)≤cf(x)、0<c<1，且固定底層f(x0)>0，反向沿遞迴k層得到f(b^k x0)≥(a/c)^k f(x0)。因 $\log_b(a/c) = p + \log_b(1/c)$，所以f(n)=Ω(n^(p+ε))，ε=log_b(1/c)>0。

這解釋為何在正且良好基底下，某些版本不另列多項式分離；本章保留標準三情形的完整假設以便直接核對。這項推導在已明列的精確冪域成立；對一般實數或整數域還要控制最後落入哪個固定區間及該區間的正下界。

本章至此已給主定理各情形的一般證明與整數延伸；下一部分的不等比例工具則明列為引用定理，不能混稱同一個證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Akra–Bazzi：不等子問題的引用定理

主定理處理同一大小 `n/b` 的 a 個子問題；Akra–Bazzi 允許多種固定比例。這是 CLRS 第三版第 4 章以外的延伸工具。本節採 MIT 6.042J 的連續形式，另以正且有上下界的基例作為充分假設。

令 `T(x)` 在 `0<=x<=x_0` 滿足 `0<d_1<=T(x)<=d_2`，其中 d_1、d_2 是固定常數；對 `x>x_0` 定義

`T(x)=sum_{i=1}^k a_i T(b_i x+h_i(x))+g(x)`。

須同時滿足：

- k 有限且固定，所有 `a_i>0`、`0<b_i<1` 為固定常數。
- `g>=0` 可微，且存在固定 C、D，使充分大 x 的 `|g'(x)|<=D x^C`；本節各例的 g 在基底區間也連續。
- 每個 `|h_i(x)|=O(x/log^2 x)`。
- `x_0` 足夠大，使每個 child argument 都在 `[0,rho*x]`，其中固定 `rho<1`。遞迴因而有定義，且會進入基例。

令唯一實數 p 滿足 `sum_i a_i b_i^p=1`，則

`T(x)=Theta(x^p (1+integral_1^x g(u)/u^{p+1} du))`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Akra–Bazzi：不等子問題的引用定理 （續）

正基例提供葉層的正成本，不能改成容許整個問題成本為零：`T≡0,g≡0` 會使右側的正 Theta 下界失效。本節引用這個定理；以下推導特徵根、積分與適用性，不把它的完整證明與主定理證明混為一談。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

取整與小擾動的界線

對固定比例 b_i，floor 和 ceiling 相對 `b_i x` 的誤差絕對值至多 1，因此符合上述擾動界。加一個固定常數也同樣符合；仍須把基底範圍選得足夠大，確保 child 真正縮小。

若差異為 `epsilon*x`，其中 epsilon>0 固定，則它不是 `O(x/log^2 x)`；這是比例改變，必須重新求特徵根。這個檢查直接關係到指數，不能以「都是二個遞迴呼叫」代替。

不同來源可能使用除法參數 `x/c_i`、`c_i>1`。它與本節乘法比例的對應是 `b_i=1/c_i`；特徵方程因而分別寫成 `sum_i a_i c_i^{-p}=1` 和 `sum_i a_i b_i^p=1`，不可在同一個例子中互換符號定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不等子問題的唯一指數與積分

依本節的乘法比例定義 `F(p)=sum_i a_i b_i^p`。每個 `b_i^p` 對 p 嚴格遞減，所以 `F` 也嚴格遞減。它在 `p->-∞` 趨近無限大，在 `p->∞` 趨近 0；由介值定理，對任何有限正係數 `a_i` 都恰有一個實根，不能只在本例先假定根存在。`F(0)=sum_i a_i`：大於 1 時根為正，等於 1 時根為 0，小於 1 時根為負。p 可以是分數；公式給的是 Theta 階，不是精確閉式解。

取

`T(x)=1` 於整個 `0<=x<=1` 基例區間；`x>1` 時 `T(x)=T(x/3)+T(2x/3)+x`。

此時 `a_1=a_2=1`、`b_1=1/3`、`b_2=2/3`、`g(n)=n`。特徵方程為

`(1/3)^p+(2/3)^p=1`。

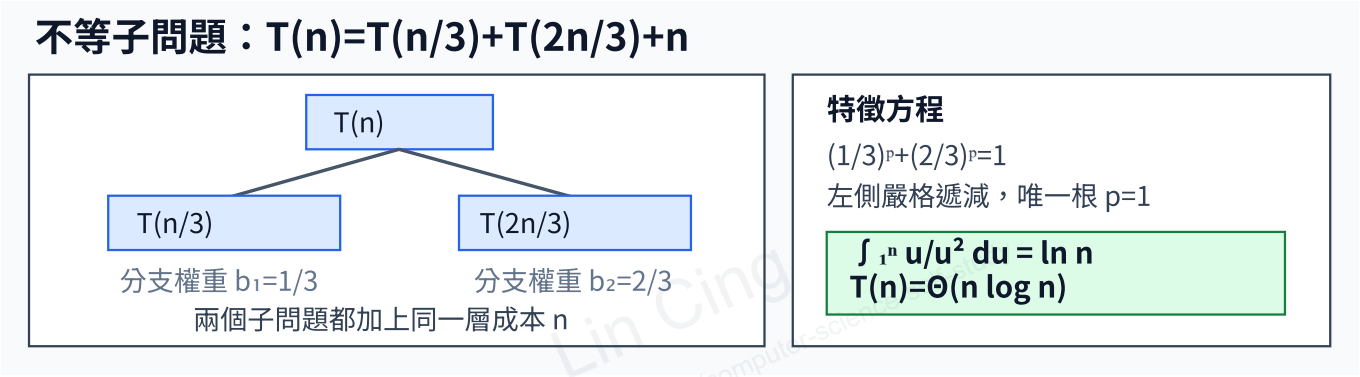
代 `p=1` 得 `1/3+2/3=1`；因左側嚴格遞減，沒有第二個根。代入積分：

`T(n)=Theta(n(1+integral_1^n u/u^2 du))`

`=Theta(n(1+ln n))=Theta(n log n)`。

這個結果不依賴某個固定 n 的枚舉。下界與上界都由一般定理同時給出。

不等子問題的唯一指數與積分 （續）



擾動過大時不能照套

考慮

$T(n)=T(n/2+n/4)+T(n/2)+n$。

若保留表面參數 $b_1=b_2=1/2$，會得到 $(1/2)^p+(1/2)^p=1$，即 $p=1$，預測 $\Theta(n \log n)$。但第一個 child 並未落在 $n/2+O(n/\log^{1+\epsilon}n)$；擾動是 $n/4$，違反小擾動條件。

這個遞迴必須先改寫成

$T(n)=T(3n/4)+T(n/2)+n$。

現在 $b_1=3/4$、$b_2=1/2$，特徵方程是

$(3/4)^p+(1/2)^p=1$。

數值解 $p\approx1.507126591639$，代回積分得 $\Theta(n^p)$，不是 $\Theta(n \log n)$。因此「看起來有兩個 1/2 子問題」不構成套用條件；必須先確認每個實際 child 大小，並核對擾動界。

主定理與 Akra–Bazzi 的判定順序

遇到遞迴時，先按模型分類，再選工具。

1. 所有子問題都是同一大小 n/b 的 a 份：先檢查主定理的三種情形與正則性。
2. 子問題有固定多種比例，例如 $n/3$ 與 $2n/3$：可用 Akra–Bazzi，先解 p 再算積分。
3. 索引只在 floor、ceiling 或 $O(n/\log^2 n)$ 的小範圍內擾動：可保留原比例；若擾動已達固定比例，必須改寫成真實的基底比例。
4. g 有負值、不符合已述平滑性條件，或基例沒有良好定義時，定理前提不足；改用展開或直接歸納，不要以公式硬套。

對 $T(n)=T(n/3)+T(2n/3)+n$，主定理的三情形不能直接表示不同 child 大小；Akra–Bazzi 正是在處理這個缺口。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 04。

M. Akra and L. Bazzi, "On the Solution of Linear Recurrence Equations," *Computational Optimization and Applications* 10(2), 1998, doi:10.1023/A:1018353700639.

K. Mehlhorn, *A Master Theorem for Recurrences*, Max Planck Institute for Informatics, 2008, Theorems 1-3.

MIT 6.042J, *Mathematics for Computer Science*, Fall 2010, § 10.4.3, Theorem 10.4.1. 官方講義. 本章採正且有界基例的充分條件。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study